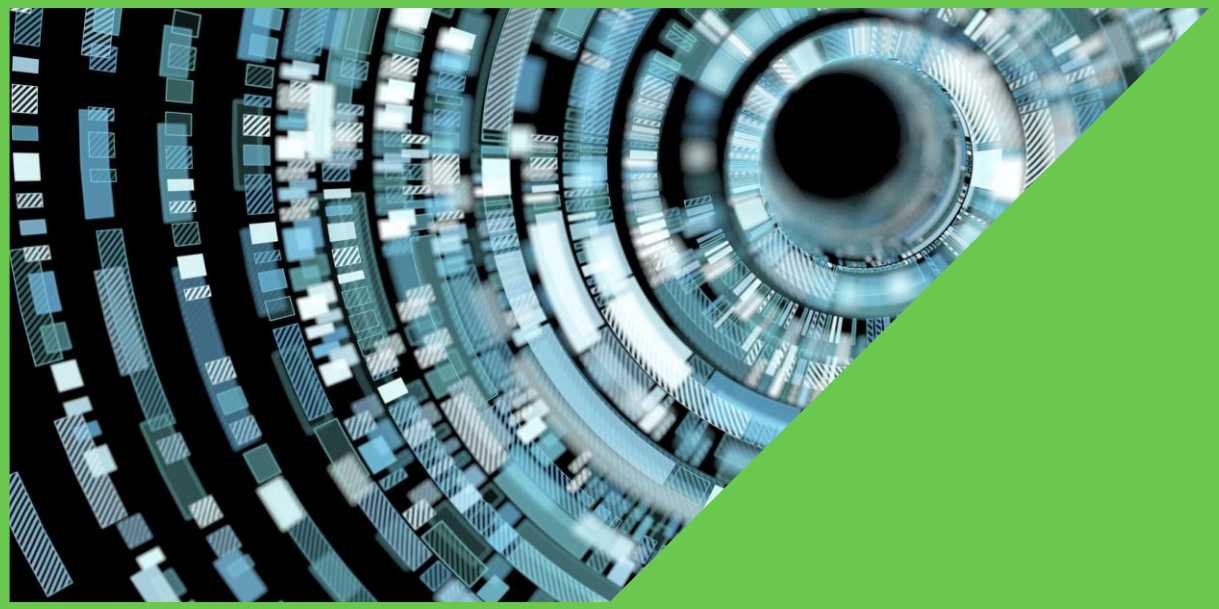
SAP Code Vulnerability Analyzer

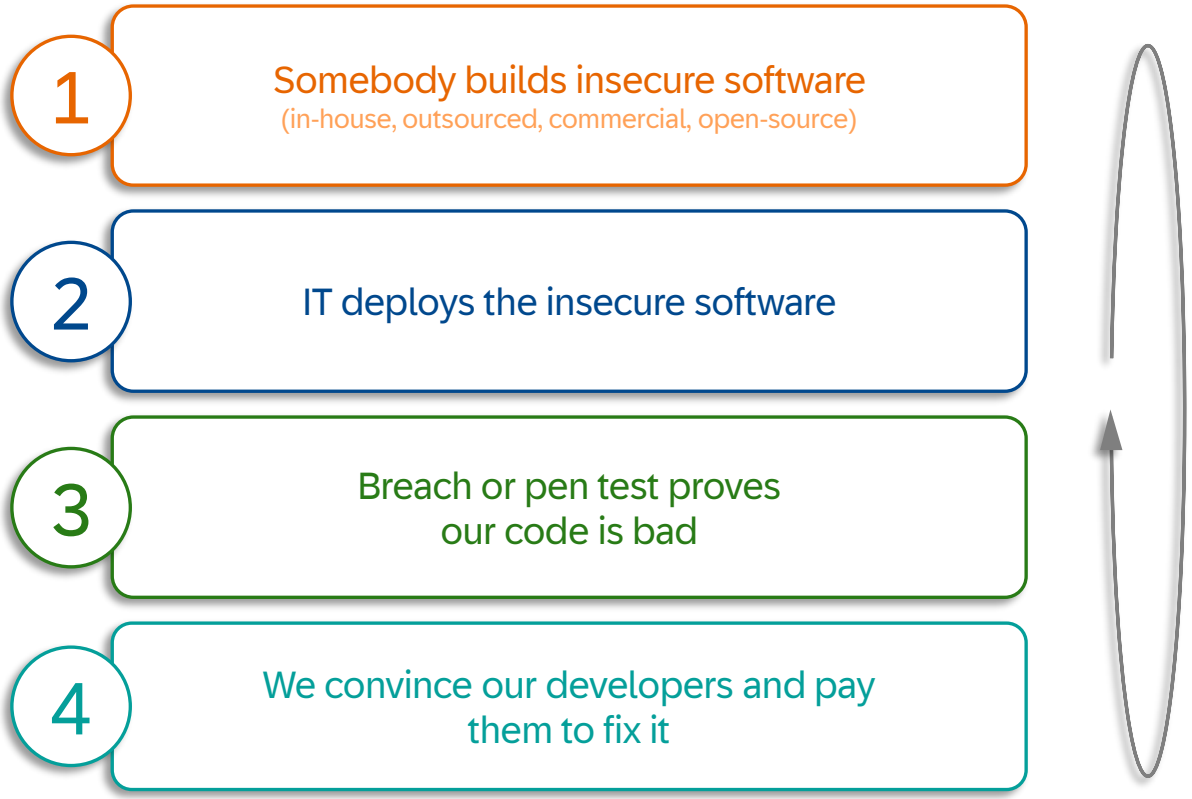
SAP BTP ABAP Product Management, SAP
October 2025



PUBLIC

OVERVIEW

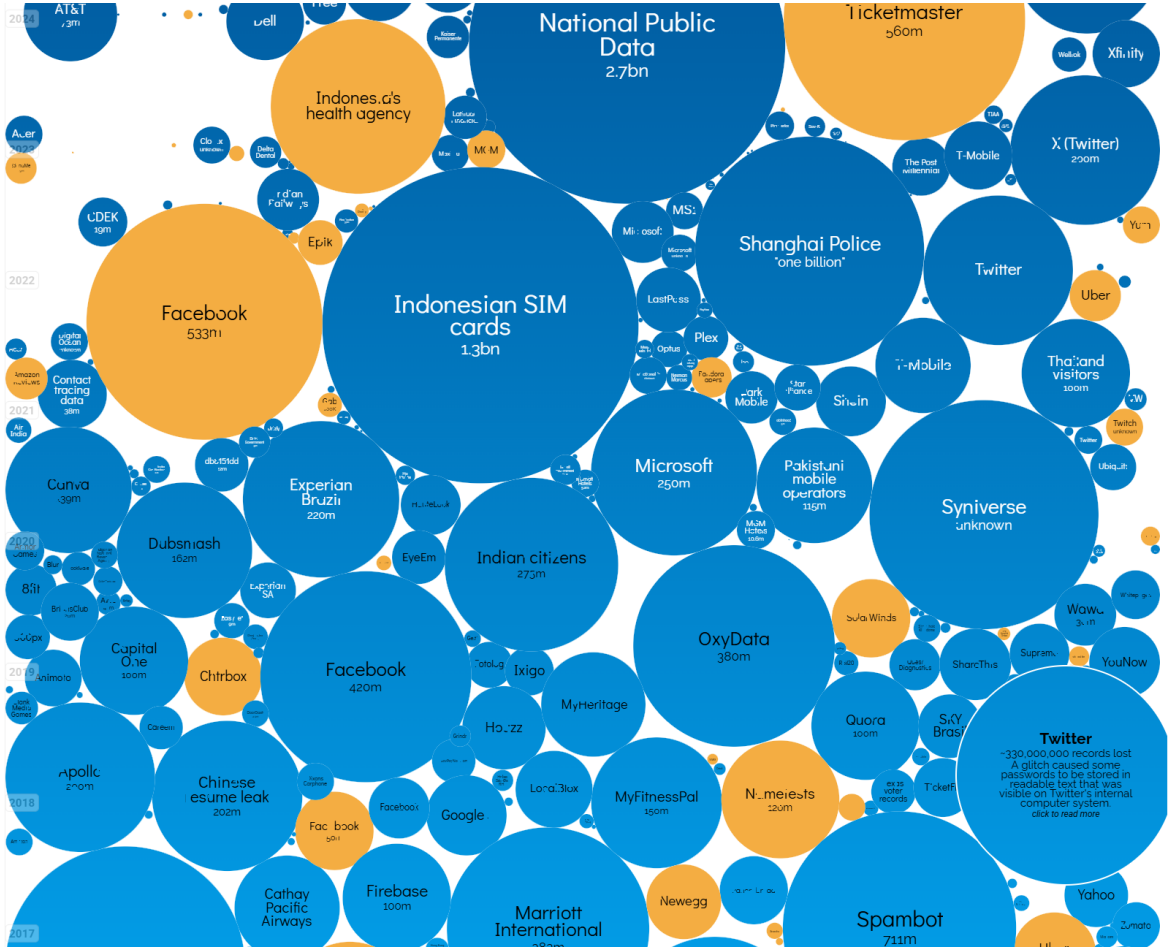
The old-fashioned approach - expensive and reactive



World’s largest data breaches and hacks

Security failures can result in:

- Negative publicity
- Brand damage
- Lost revenue
- Legal consequences
- Penalties



Source: http://www.informationisbeautiful.net/visualizations/worlds-biggest-data-breaches-hacks/

The challenge of security

In order to **secure** an application, **all** of its components, functions, infrastructure and the related **threats** must be understood.

In order to **break** an application, only **one flaw** in any of its components/functions or the infrastructure may be enough.

Problem

- Each new technology adds the risk of new vulnerabilities.
- Firewalls, intrusion detection systems, signatures and encryption are not sufficient to make applications secure.

Application security testing solutions at SAP

Dynamic application security testing

Find vulnerabilities in the running application

- Manual application penetration testing
- Automated application vulnerability scanning

Static application security testing

Find vulnerabilities analyzing the sources

- Manual source code review
- Automated source code analysis

→ SAP CODE VULNERABILITY ANALYZER (CVA)

“Finding security issues at design time instead of in production is easier and less expensive!”

SAP uses CVA to scan its ABAP code

1 SAP on-premise software development systems
~8,500

2 SAP cloud development systems
~ 500

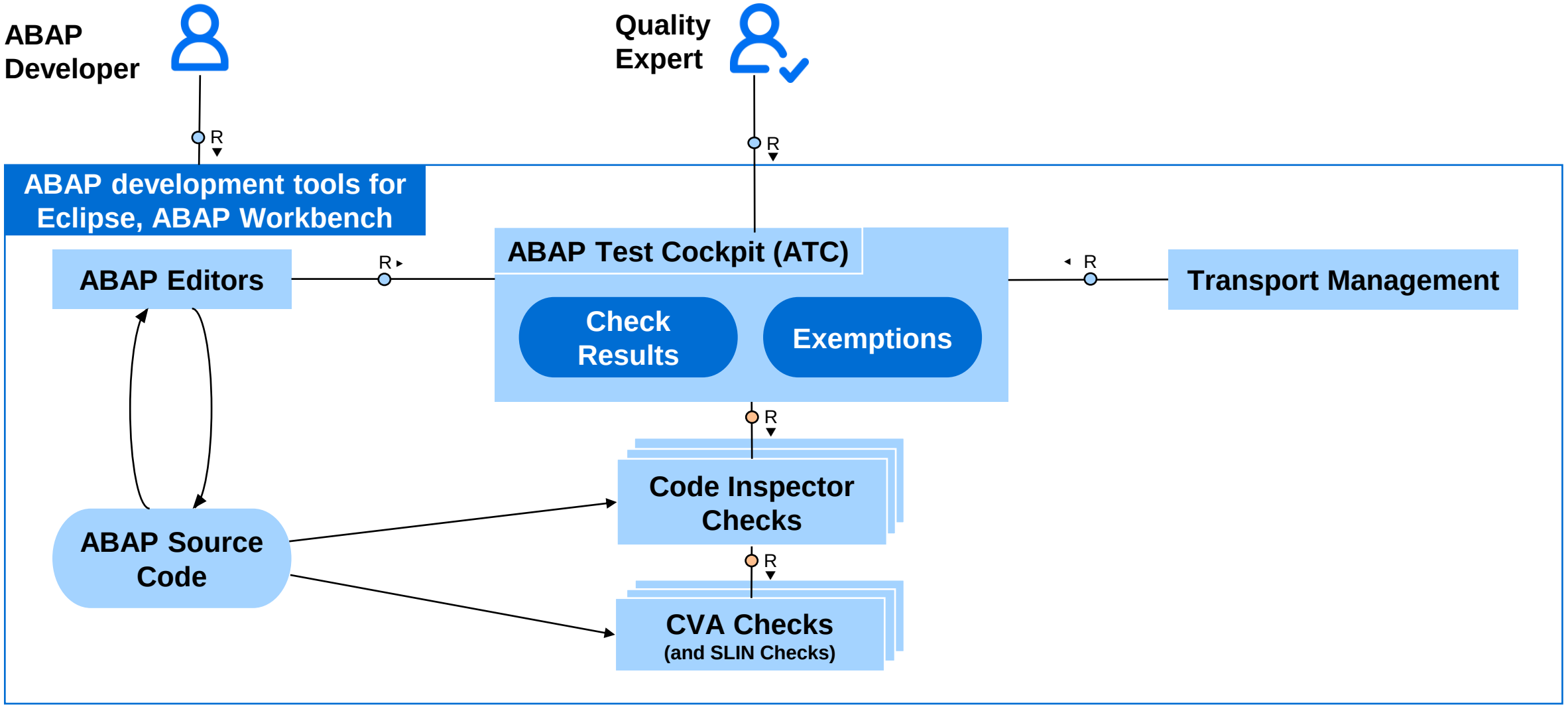
3 SAP internal business systems
~40

SAP development runs security tests on all SAP applications and code delivered by SAP.

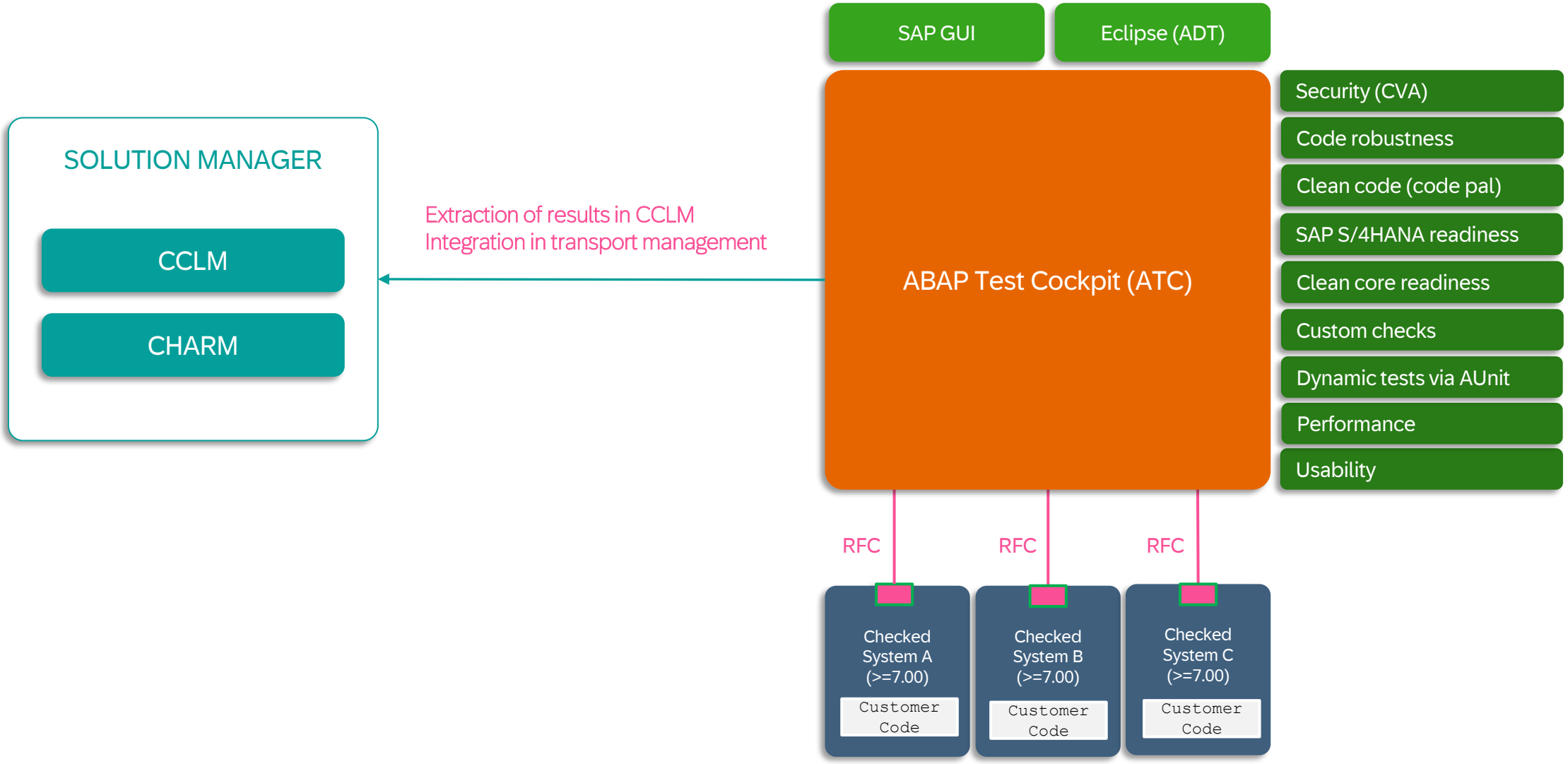
SAP development uses SAP CVA to scan more than 500 million lines of ABAP code before delivery to our customers.

SAP Code Vulnerability Analyzer

Architecture



SAP Code Vulnerability Analyzer
as a part of the ABAP Test Cockpit (ATC)



DEMO

Introductory example: SQL injection

```
3  *&---------------------------------------------------------------------*
4  *& Demo Program for an SQL Injection
5  *&---------------------------------------------------------------------*
6  REPORT zr_sql_injection.
7
8  PARAMETERS: street  TYPE zemployees-street    LOWER CASE,
9              zipcode TYPE zemployees-zipcode   LOWER CASE,
10             city    TYPE zemployees-city      LOWER CASE,
11             phone   TYPE zemployees-phone_ext.
12
13 DATA: set_expr TYPE string,
14       user     TYPE xubname.
15
16 IF street IS NOT INITIAL.
17   set_expr = set_expr && `STREET     = '` && street   && `'`.
18 ENDIF.
19 IF zipcode IS NOT INITIAL.
20   set_expr = set_expr && `ZIPCODE    = '` && zipcode && `'`.
21 ENDIF.
22 IF city IS NOT INITIAL.
23   set_expr = set_expr && `CITY       = '` && city    && `'`.
24 ENDIF.
25 IF phone IS NOT INITIAL.
26   set_expr = set_expr && `PHONE_EXT = '` && phone   && `'`.
27 ENDIF.
28
29
30 IF set_expr IS NOT INITIAL.
31   user = cl_abap_syst=>get_user_name( ).
32   UPDATE zemployees
33      SET (set_expr)
34    WHERE userid = user.
```

Possible SQL injection (SET clause)

INPUT FOR **STREET**:

Long lane' salary = '7897

SET_EXPR:

STREET = 'Long lane'
salary = '7897'

```
...
SET STREET = 'Long lane'
    salary = '7897'
```

How the code analysis works

```
3  *&---------------------------------------------------------------------*
4  *& Demo Program for an SQL Injection
5  *&---------------------------------------------------------------------*
6  REPORT zr_sql_injection.
7
8  PARAMETERS: street  TYPE zemployees-street    LOWER CASE,
9              zipcode TYPE zemployees-zipcode   LOWER CASE,
10             city    TYPE zemployees-city      LOWER CASE,
11             phone   TYPE zemployees-phone_ext.
12
13 DATA: set_expr TYPE string,
14       user     TYPE xubname.
15
16 IF street IS NOT INITIAL.
17   set_expr = set_expr && `STREET    = '` && street  && `'`.
18 ENDIF.
19 IF zipcode IS NOT INITIAL.
20   set_expr = set_expr && `ZIPCODE   = '` && zipcode && `'`.
21 ENDIF.
22 IF city IS NOT INITIAL.
23   set_expr = set_expr && `CITY      = '` && city    && `'`.
24 ENDIF.
25 IF phone IS NOT INITIAL.
26   set_expr = set_expr && `PHONE_EXT = '` && phone   && `'`.
27 ENDIF.
28
29
30 IF set_expr IS NOT INITIAL.
31   user = cl_abap_syst=>get_user_name( ).
32   UPDATE zemployees
33      SET (set_expr)
34    WHERE userid = user.
```

1. There is an input field

3. There is a data flow between the input field and the dangerous statement

2. There is a potentially dangerous statement

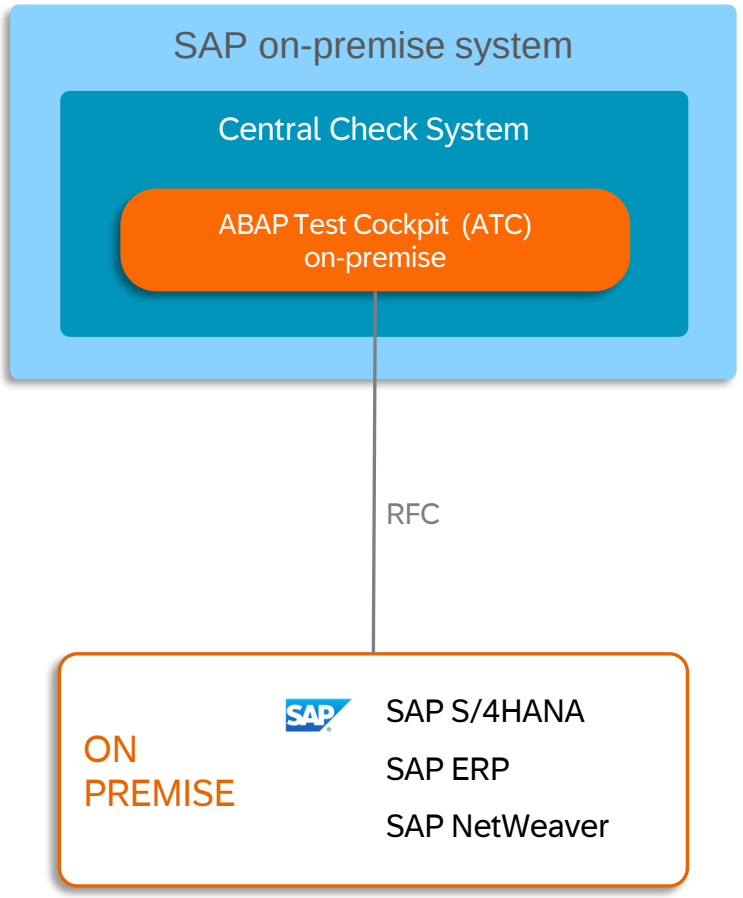
THE CODE ANALYZER LOOKS FOR POTENTIALLY VULNERABLE STATEMENTS,
WHERE THE INPUT COMES FROM UNTRUSTED SOURCES.

ONLY SUCH OCCURRENCES ARE REPORTED!

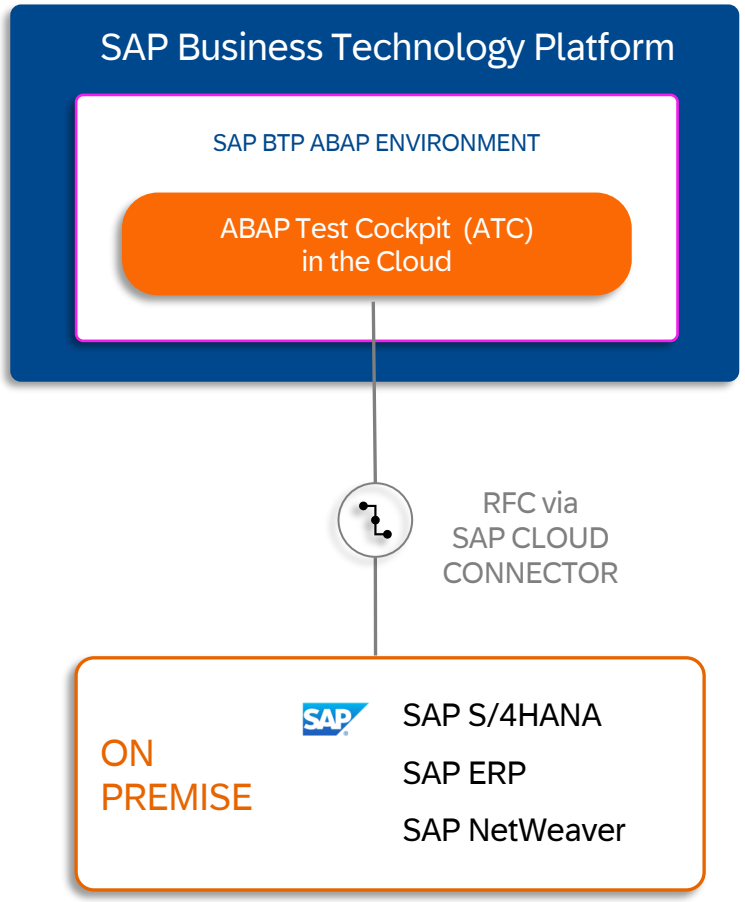
TECHNICAL INFRASTRUCTURE

Two options for technical setup of SAP Code Vulnerability Analyzer

ABAP test cockpit on-premise



ABAP test cockpit on SAP BTP



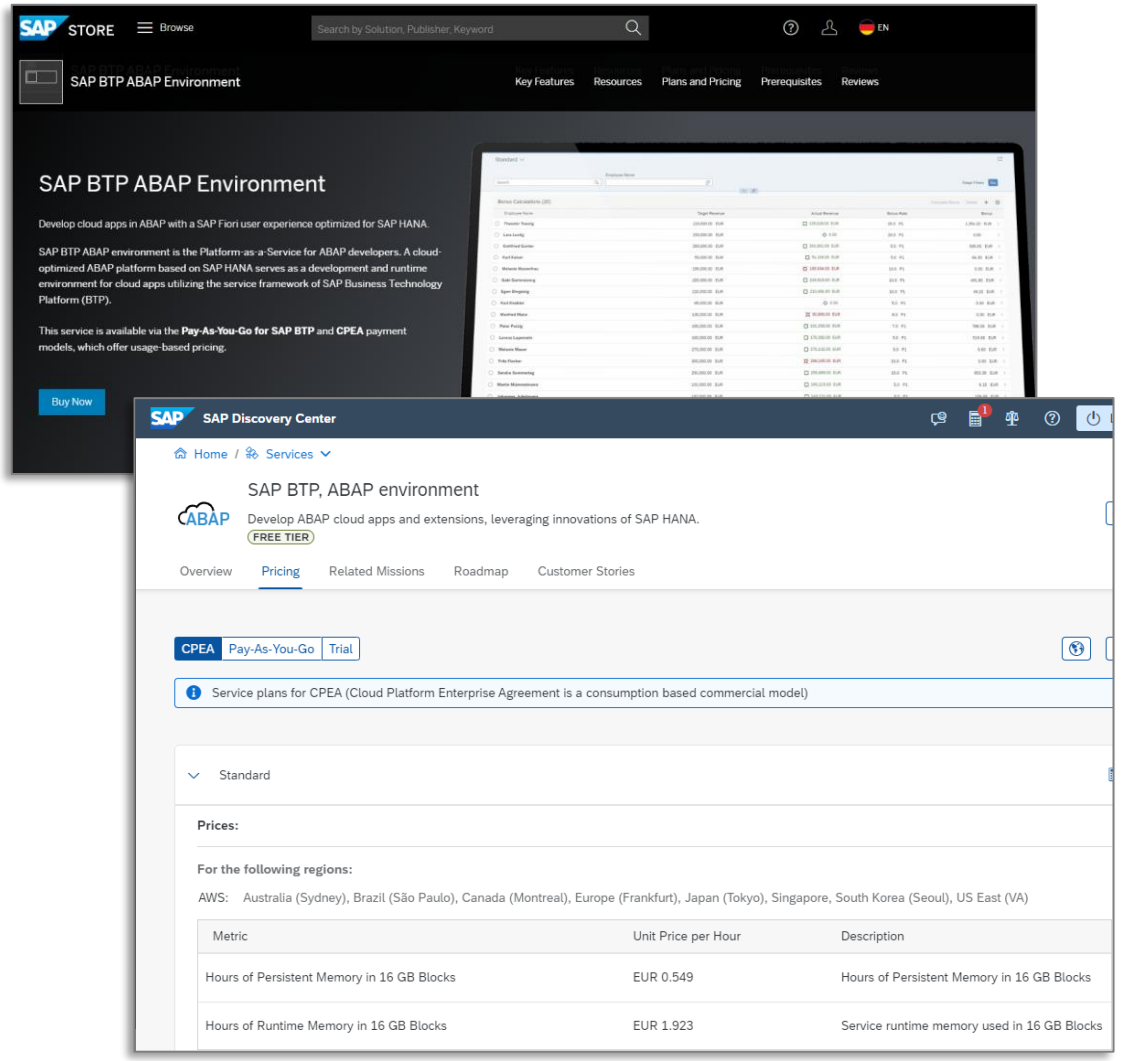
Pricing information for ABAP test cockpit on SAP BTP

How to get SAP BTP ABAP Environment

- SAP Store
- SAP Discovery Center
- SAP BTP pricing models: CPEA and Pay-As-You-Go
- Also as SAP BTP Free Tier model
- System hibernation to suspend or stop system to reduce the costs for temporarily unused systems
- See SAP Community for SAP BTP ABAP Environment

CONFIGURATION FOR CLOUD ATC

- ✓ 1 block ABAP runtime (16GB)
- ✓ 2 blocks SAP HANA persistency (2 x 16GB = 32GB)



Pricing information for CVA on-premise

License information for CVA				
Material	Item	Coding	Blocks of	Pricing metric
7019502	SAP CVA, SAP S/4HANA on-premise	ABAP	5	Users*
8011369	SAP CVA, SAP S/4HANA Cloud Private Edition	ABAP	5	Users

* A user is anyone who triggers a CVA run or who reviews the results.

The price is capped at 100 users. If you have more than 100 users then the rest are free-of-charge

ABAP Test Cockpit on-premise - Remote code analysis

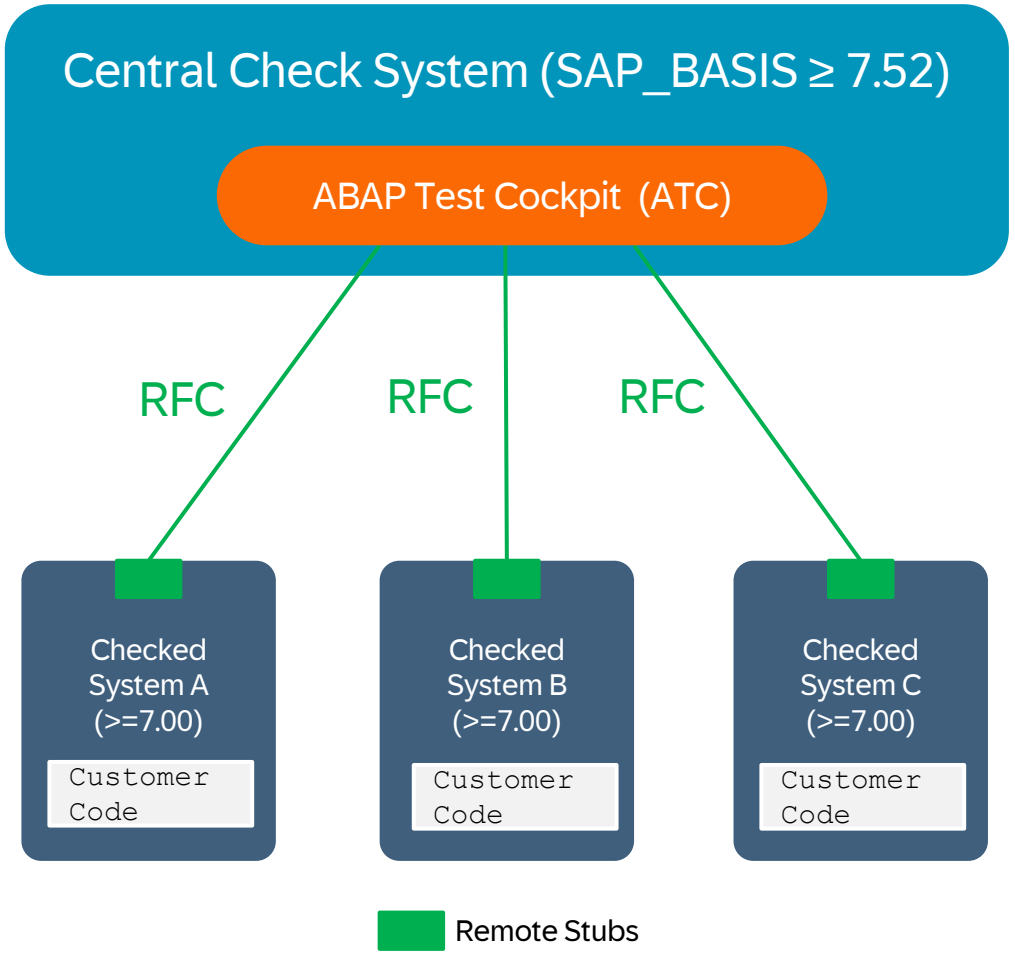
One central ATC system (SAP_BASIS >=7.52) for all security checks in your system landscape

- Remote stubs return a model from custom code
- Check logic is executed on central system
- Check variant is maintained in central system
- New checks are installed on central system
- Exemptions are stored on central system
- Integrated in the development system

→ Minimized administration efforts

→ One quality standard for your whole system landscape

More about [Remote code analysis with ATC]



Priority of each check can be adjusted to match the requirements

Ability to control the priority of every single finding

Take your own risk and security requirements into account

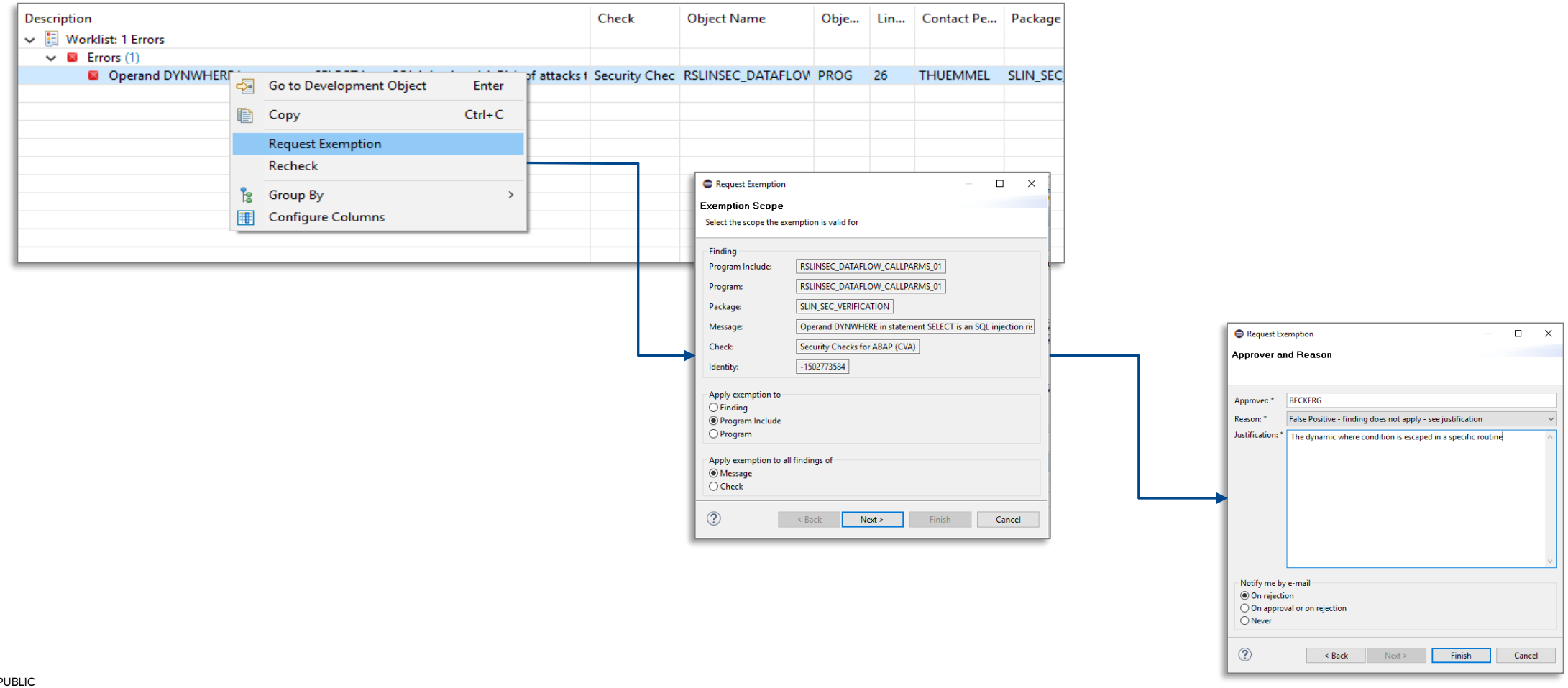
Possibility of a phased approach, spreading out the workload for developers and so increasing acceptance for security testing

Code Inspector: Message Priorities

Code Inspector Che...	Check/Message Text	Message Co...	S.	C.	Exception
	Security Checks				
	Security Tests in Extended Program Check				
	Hard coded username	0821			Not Possible
	Possible SQL injection (WHERE condition)	1101			Not Possible
	Possible Directory Traversal	1104			Not Possible
	Possible system command injection (addition FILTER)	1106			Not Possible
	Possible ABAP command injection	1108			Not Possible
	Possible SQL injection (SET clause)	1112			Not Possible
	Possible SQL injection (column names)	1114			Not Possible
	Possible SQL injection (GROUP BY clause)	1116			
	Possible SQL injection (HAVING clause)	1117			
	Possible SQL injection (table name when reading)	1118			Not Possible
	Possible SQL injection (table name when writing)	1120			Not Possible
	Possible SQL injection via Object Services	1122			Not Possible
	Possible Directory Traversal via File Utilities class	1124			Not Possible
	Potential Directory Traversal due to insecure parameters	1126			Not Possible
	Possible SQL injection via ADBC (DDL)	1128			Not Possible
	Possible SQL injection via ADBC (DML)	1130			Not Possible
	User controlled dynamic CALL FUNCTION	1140			Not Possible
	User controlled dynamic program unit call	1141			Not Possible
	User controlled dynamic CALL TRANSACTION	1142			Not Possible
	User controlled dynamic LEAVE TO TRANSACTION	1143			
	Usage of an obsolete Escaping method	1150			Not Possible
	AUTHORITY-CHECK without processing of sy-subrc	1160			Not Possible
	Call of System Function CALL 'SYSTEM'	1170			Not Possible
	AUTHORITY-CHECK for specified User	1180			Not Possible
	AUTHORITY-CHECK for SY-UNAME	1181			
	SUBMIT for specified User	1185			Not Possible
	The dynamic WHERE condition allows a potential code injection.	1190			Not Possible

ABAP Test Cockpit on-premise - Exemptions

- Using exemptions workflow for dealing with false positives



ABAP Test Cockpit on-premise – Baseline

Baseline concept – Focus on findings in new or recently changed code

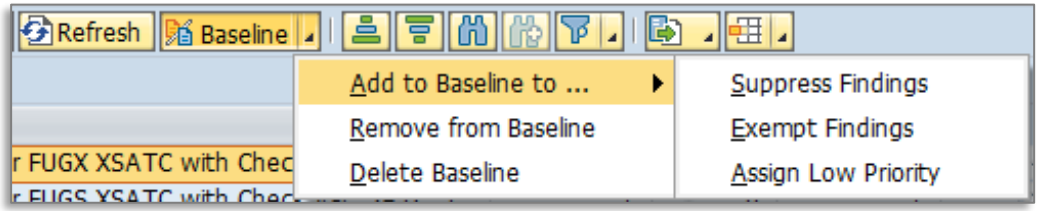
- Transfer individual ATC results into the baseline
- Options
  - suppress all findings (exclude from the ATC result)
  - indicate the findings as exempted
  - assign the low priority to the findings
- The baseline is effective as long as the related code sections remain unchanged

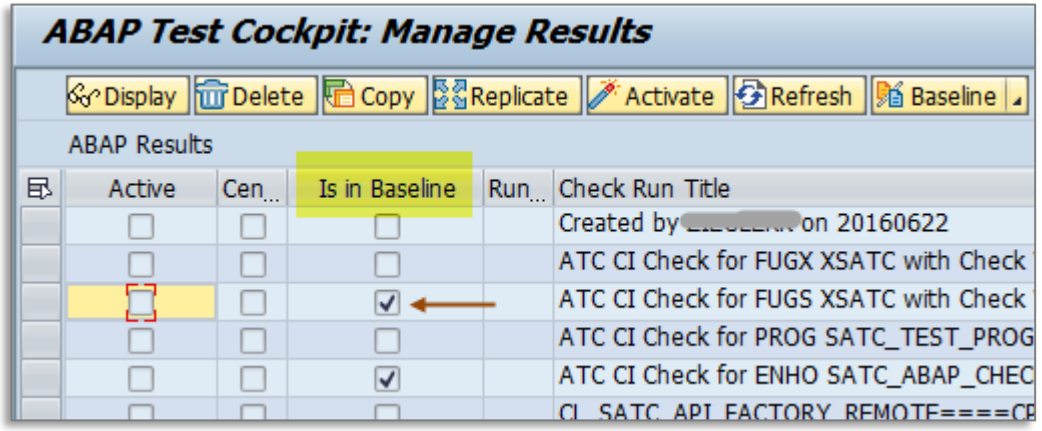
Baseline activities

- Adding/removing individual check results
- Deleting baseline (reset the test system to original state)

More about [Working with Baseline in ATC]

BASELINE MANAGEMENT



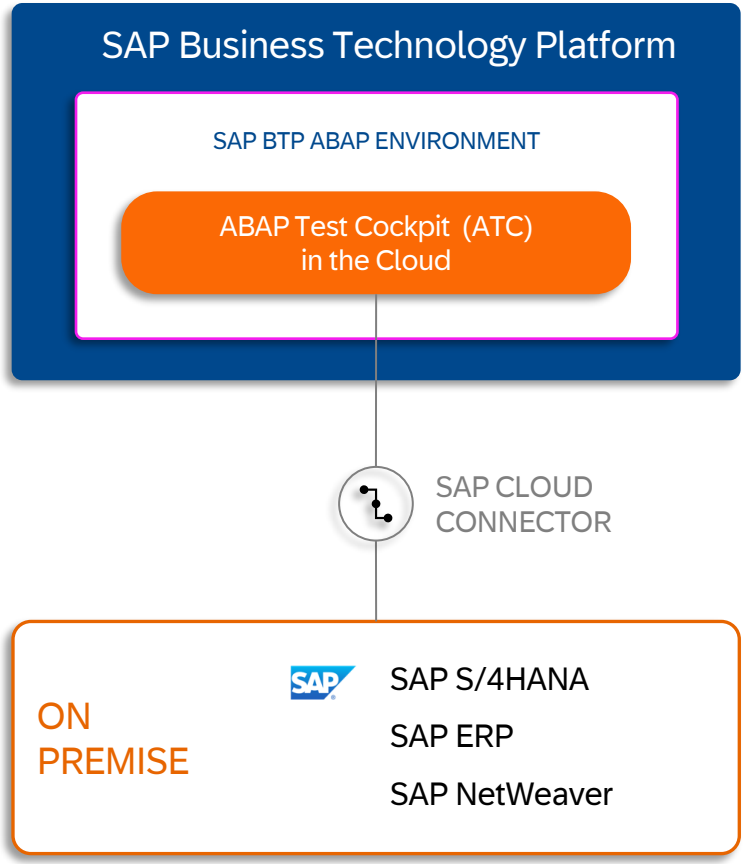


Code Vulnerability Analyzer (CVA) in ABAP Test Cockpit on SAP BTP

- CVA is based on ABAP test cockpit in the Cloud
- CVA in SAP BTP ABAP Environment can be used without additional license fee*

More: [ABAP Test Cockpit in the Cloud – What is already possible]

* Only entitlement for SAP BTP ABAP Environment is required

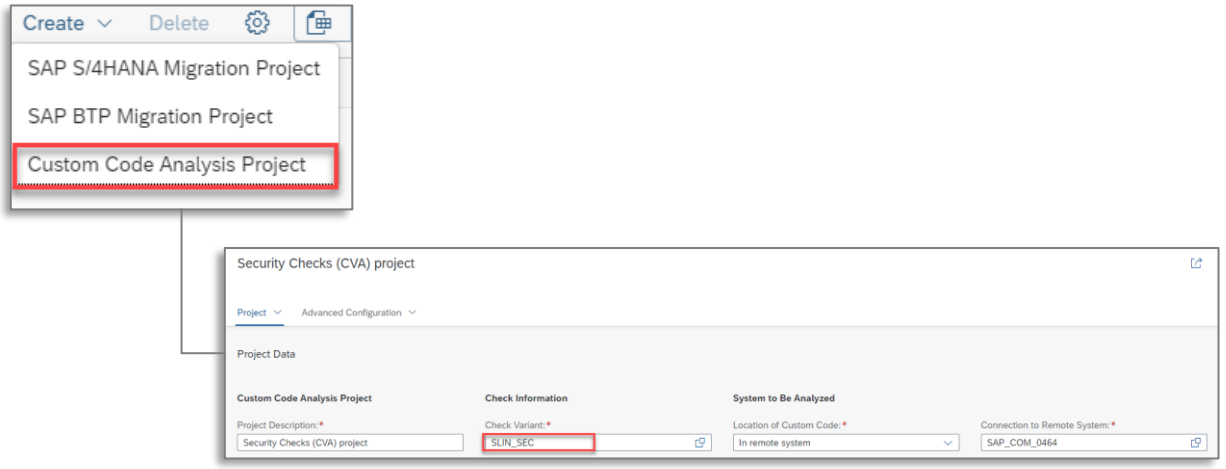


Execute CVA checks via SAP BTP ABAP Environment in on-premise systems

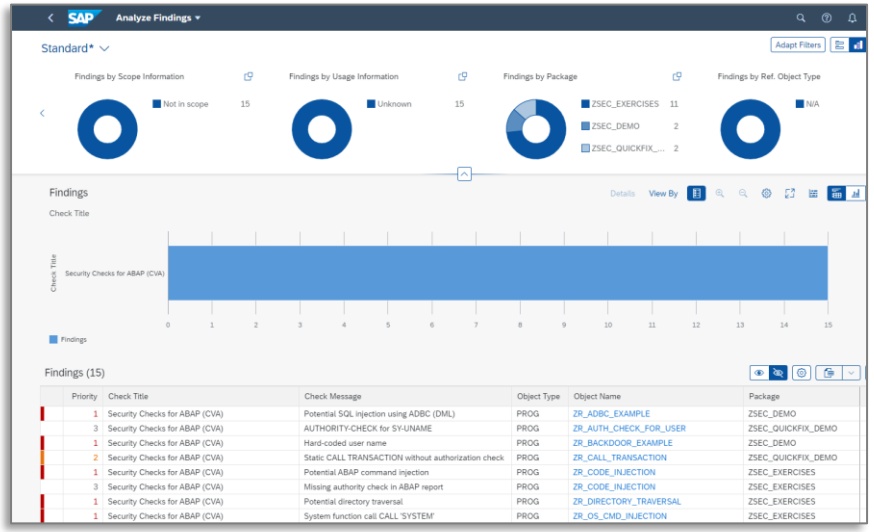
- Create Custom Code Analysis Project and use SLIN_SEC check variant
- Consider usage data to focus your analysis on security vulnerabilities in used code

More information:
[ATC in the cloud for on-premise developments]

RUN CVA CHECKS



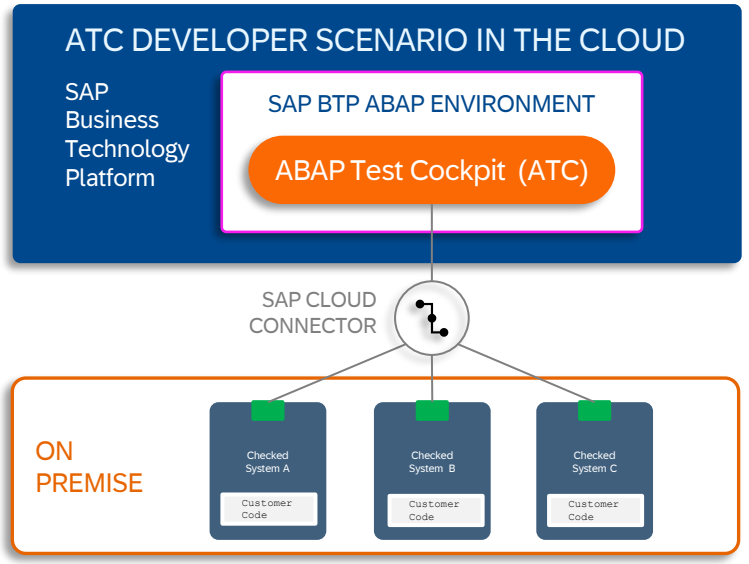
CVA RESULTS ANALYSIS



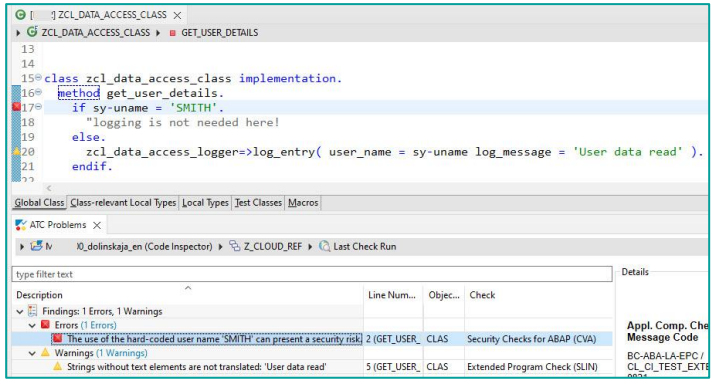
Use CVA checks in SAP BTP ABAP Environment as on-premise developer

USING REMOTE ATC IN THE SAP BTP ABAP ENVIRONMENT DURING DEVELOPMENT IN ON PREMISE SYSTEM

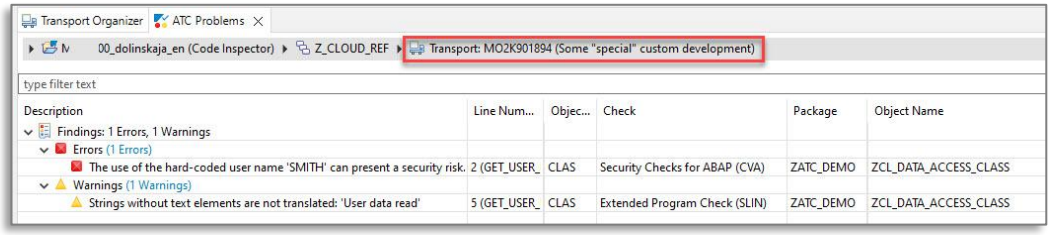
- Execute ATC (CVA) checks
- View ATC (CVA) check results
- Check transports before release
- Correct findings
- Request exemptions



EXECUTE ATC/CVA CHECKS



CHECK TRANSPORT BEFORE RELEASE



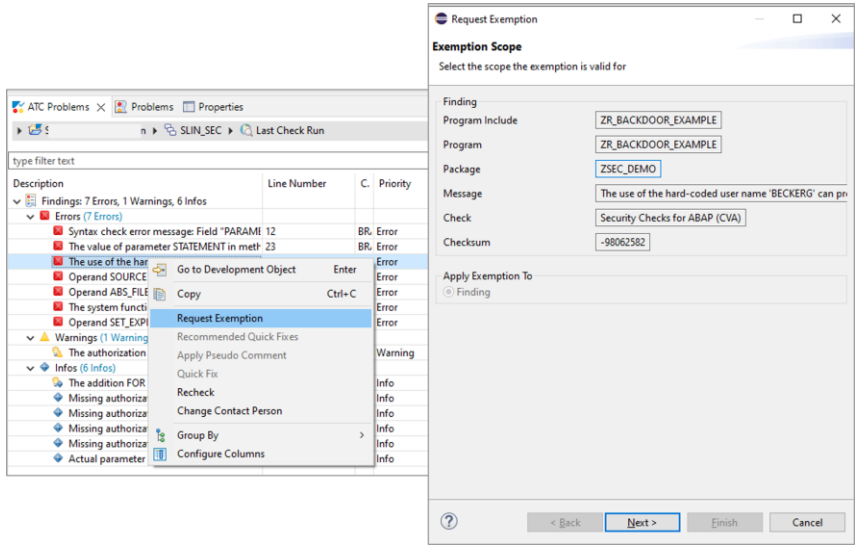
More information:
[Using ATC in the cloud as developer in an on-premise system]

Request exemptions for CVA findings in SAP BTP ABAP Environment for on-premise developments

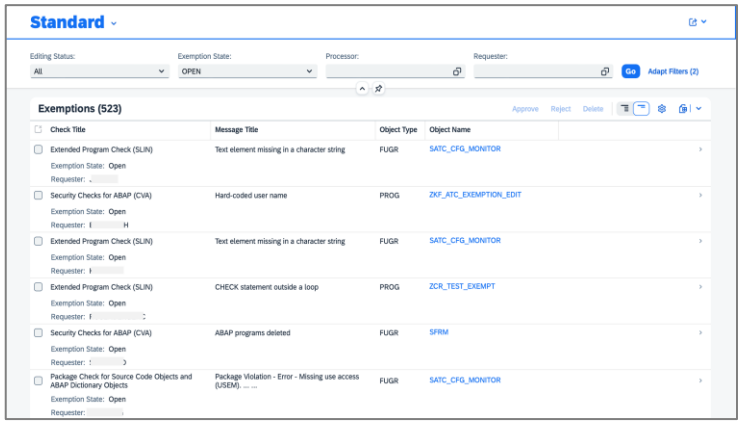
EXEMPTIONS WORKFLOW USING ATC IN THE CLOUD DURING ON-PREMISE DEVELOPMENT

- Create an ATC exemption in an on premise system for a CVA finding using ABAP development tools for Eclipse
- Approve/reject an ATC exemption using the new SAP Fiori-based approve ATC Exemptions app in SAP BTP ABAP Environment

REQUEST EXEMPTION IN ABAP DEVELOPMENT TOOLS FOR ECLIPSE



PROCESS EXEMPTION IN SAP FIORI-BASED EXEMPTION APP



More information:
[How to work with exemptions using ATC in the cloud during on-premise development]

Custom code analysis features comparison

	ATC ON-PREMISE	ATC IN SAP BTP
Required system	ATC check system **(SAP_BASIS >= 7.52)**	SAP BTP ABAP environment
Application	SAP GUI-based	SAP Fiori & Eclipse-based
Remote connectivity	via RFC	via RFC and SAP Cloud Connector
Code Vulnerability Analyzer (CVA) security checks	additional license	no additional license
Usage data consideration	✗	✓
Always use newest checks & tool features	only via new releases	upgrade every 3 months
Findings reported in the prioritized worklist	✓	✓
Analysis of findings: source code navigation, documentation	✓	✓
Robust exemption process	✓	✓
Baseline to exclude legacy findings	✓	✓
Developer scenario	✓	✓
Custom check variants incl. custom checks	✓	✓
Transport management und CHARM integration	✓	✓
Execute runs via REST service (CI pipeline)	✗	✓
System maintenance	Customer managed	SAP managed

TYPES OF SECURITY CHECKS

Overview of available security checks

- Web exploitability
- Backdoors
- Sensitive data access
- Missing or incorrect authorization checks
- SQL injections (ABAP SQL and ADBC)
- ABAP code and call injections
- OS command injections
- Directory traversal

Overview of the available checks - SQL injection (ABAP SQL)

MANIPULATION OF DYNAMIC ABAP SQL

Potential manipulation of the dynamic WHERE condition (1101)

Potential manipulation of the dynamic WHERE condition with possible expansion to other tables (1102)

Potential manipulation of a dynamic WHERE condition using the parameter I_FILTER of the object services method CREATE_QUERY (1122)

Potential manipulation of the dynamic WHERE condition of a wrapped SQL statement (110A)

Potential manipulation of the SET clause in the statement UPDATE (1112)

Potential read performed on an illegal database table in a SELECT statement (1118)

Potential unauthorized access to a database table in a wrapped SELECT statement (110B)

Potential read performed on an illegal database table in a modifying ABAP SQL statement (1120)

Potential read performed on invalid table columns (1114)

Potential use of illegal columns in a dynamic GROUP BY clause (1116)

Potential use of illegal columns in a dynamic HAVING clause (1117)

Potential read performed using an invalid secondary database connection in an ABAP SQL statement (1121)

Potential manipulation of a dynamic database query (1123)

Overview of the available checks - SQL injection (native SQL)

MANIPULATION OF DYNAMIC NATIVE SQL

Potential injection of harmful SQL statements or clauses in execution of DDL statements in ADBC (1128).

Potential injection of harmful SQL statements of clauses in execution of DML statements in ADBC (1130).

Potential injection of malicious SQL statements or clauses when calling an appropriate API (11D1)

Overview of the available checks - ABAP code injection

MANIPULATION OF DYNAMIC ABAP CODE

Potential injection of harmful code in the statements INSERT REPORT and GENERATE SUBROUTINE POOL (1108)

Potential injection of harmful code via call of RFC-enabled function module RFC_ABAP_INSTALL_AND_RUN (1109)

Potential manipulation of the dynamic WHERE condition in an internal table (1190)

Potentially important programs deleted from the ABAP repository (1110)

Potentially important programs replaced in the ABAP Repository (1111)

Overview of the available checks - ABAP call injection

MANIPULATION IN DYNAMIC CALLS

Dynamic CALL TRANSACTION without check on valid transaction names (1142)
Dynamic CALL TRANSACTION without authorization check (114E, 114F, 114H)
Dynamic CALL TRANSACTION with potentially incomplete authorization check (114G, 11GI)
Potential call of an unwanted transaction using the statement LEAVE TO TRANSACTION (1143)
Potential call of an illegal program using the statement SUBMIT (1141)
Potential call of invalid function module using RFC (1140).
UI-driven or RFC-driven dynamic call of a function module (1144)
RFC call with dynamic destination (1145)
C function called with names called as potential user input (1171)

Overview of the available checks - OS command injection

INJECTION OF OS COMMANDS

Statement CALL 'SYSTEM' used (1170)

Statement SYSTEM-CALL used (1172)

Calling a system function using the statement CALL cfunc (1173)

Use of a non-secure parameter of an SXPG function module (1178)

Potential manipulation of the FILTER addition for statement OPEN DATASET (1106)

FILTER addition of the statement OPEN DATASET used (1107)

C function called with names called as potential user input (1171)

Overview of the available checks - directory traversal

ACCESS TO NON-PERMITTED FILES

Potential manipulation of the file name in the statement OPEN DATASET or DELETE DATASET (1104)

Potential manipulation of the file name in the method CREATE_UTF8_FILE_WITH_BOM of the class CL_ABAP_FILE_UTILITIES (1124)

Non-secure parameter of the function module FILE_GET_NAME used (1126)

Overview of the available checks - missing or incorrect authorization checks

INSUFFICIENT AUTHORIZATION CHECKS

SY-SUBRC not evaluated after the statement AUTHORITY-CHECK (1160)

Return value (for example, SY-SUBRC) not evaluated after a security-relevant method was called (1161)

Return value (for example, SY-SUBRC) not evaluated after a security-relevant subroutine was called (1162)

Potential security-relevant procedure without registration (1163)

Return value (for example, SY-SUBRC) not evaluated after a security-relevant local procedure was called (1164)

Return value (for example, SY-SUBRC) not evaluated after a security-relevant function was called (1165)

AUTHORITY-CHECK with explicitly specified user name (1180, 1181)

Static CALL TRANSACTION with missing or incomplete authorization check (114A, 114B, 114C, 114D)

Potentially missing authorization check in a report (11A1)

Potentially missing authorization check in an RFC function module (11A2)

Potentially missing authorization check in a BOR object (11A3)

Potentially missing authorization check in an OData service (11A4, 11A5)

Potentially missing authorization check in a RAP BDEF implementation (11A6, 11A7)

Overview of the available checks - backdoors

POTENTIAL BACK DOORS

Hard-coded user name, possibly from undeleted test code or an indication of a back door (0821)

Hard-coded host name, possibly from undeleted test code or an indication of a back door (11S1)

Hard-coded system ID, possibly from undeleted test code or an indication of a back door (11S2)

Hard-coded client, possibly from undeleted test code or an indication of a back door (11S3)

System variable sy-xxxx compared with a hard-coded value from forgotten test code or that could indicate a back door (11S4).

Undocumented profile parameter used (11SP)

Procedure call with hard coded password (11K1)

Field with hard coded password (11K2)

Field with Base64-encoded password (11K3)

HTTP Authentication with Hard-Coded Logon Data (11K4)

Overview of the available checks - web exploitability

ATTACKS USING WEB TECHNOLOGIES

Potential risk of cross-site scripting (1132)
Potential reflected cross-site scripting (1134)
Potentially manipulated HTML page in HTML control (1135)
Potential unvalidated URL redirect (11P1)
Obsolete escape method used (1150)
Missing Content Check During HTTP Upload (11F1)
Potential reverse tabnabbing (11R1)
Potential server-side request forgery using dynamic URL (11R4)
Potential server-side request forgery using dynamic destination (11R5)
Security Checks for BSP (CVA)

Overview of the available checks - sensitive data access and other checks

SENSITIVE DATA ACCESS AND FURTHER CHECKS

Read performed on sensitive database table/view/CDS (11G0)
Read performed on sensitive released database table/view/CDS (11G2)
Write performed on sensitive database table (11G1)

Statement COMMUNICATION used (11C1)
Using FTP function modules (11C2)
Invalid validation with class CL_ABAP_DYN_PRG (11N1)
Invalid validation (tautological value check) with class CL_ABAP_DYN_PRG (11N3)

SUMMARY AND OUTLOOK

CVA on SAP BTP - the benefits

CVA on SAP BTP is always up-to-date so it has the latest checks

CVA on SAP BTP does not require a CVA license. It requires only SAP BTP ABAP Environment which is generally cheaper

CVA on SAP BTP allows the analysis of usage data

The setup time for CVA on SAP BTP is shorter

Q: CVA on SAP BTP runs on SAP BTP ABAP Environment. What can I use SAP BTP ABAP Environment for apart from CVA?

A: The Custom Code Migration app runs on SAP BTP ABAP Environment. Use cases include: Custom Code Analysis für SAP S/4HANA, SAP BTP ABAP Environment Migration, Cloud ATC.
SAP BTP ABAP Environment can also be used to develop ABAP custom code on SAP BTP or for innovative use cases that integrate with SAP BTP Services (AI, Machine Learning...) or partner SaaS applications

SAP BTP Discovery Center is a good source of information for licensing and pricing:
https://discovery-center.cloud.sap/search/BTP

SAP BTP Discovery Center is a good source of information for licensing and pricing:
https://discovery-center.cloud.sap/search/BTP

CVA - the competition

The ABAP language developers are at SAP

At SAP we use CVA to check our code

Installation and integration: CVA is already in the Cloud as part of SAP BTP ABAP Environment or in SAP NetWeaver on premise - it just needs to be activated.
It is not a solution running elsewhere, say, on a Java engine with all the overhead that this involves?

How does your solution handle updates? CVA in the Cloud is automatically up-to-date.
CVA on-premise just needs the latest SP of the latest SAP NetWeaver release

The *number* of checks is irrelevant – you need the *right* checks → fewer false positives

Small company risk: Will the company still be around in 5 years' time?

Custom code security services - security analysis for custom code

Identify security vulnerabilities in your own ABAP code.
Classify and rate individual security vulnerabilities using SAP tools: ABAP Test Cockpit (ATC) and SAP Code Vulnerability Analyzer

YOUR CHALLENGES

You use your own programs in your own SAP system

Security of custom developments assessed in the light of

- privacy policy
- compliance guidelines
- Vulnerability to cyber attacks that make use of malicious code

The investigation of potential security vulnerabilities exceeds existing financial budget and time frame

OUR SOLUTION

First of all clarify the requirements and configuration

Workshop dealing with the SAP custom code security tools ATC & Code Vulnerability Analyzer

Security analysis of custom development

Additional workshops

- **Management workshop** to present results
- **Developer/quality manager workshop** to present the results and explain the most common weaknesses identified with the help of selected customer examples

YOUR BENEFITS

Overview of vulnerabilities in your own ABAP code

Qualitative assessment of security of your own development

Valid basis for planning measures to be taken

FEATURES

Remote workshops

Remote activities

(Optional) coaching to remedy the weaknesses identified

WORK REQUIRED

10 days

INCREASED SECURITY FOR YOUR APPLICATIONS

Reduced false-positive rate by data flow analysis

Supports automation requirements by quality assurance teams

Priority of each check can be adjusted to match the requirements

Extensive documentation to support developers in fixing issues found

Integrated into standard ABAP development infrastructure to enable easy consumption by developer

Supports exemption workflows to ease handling of false-positives

Further information

SAP CODE VULNERABILITY ANALYZER

- SAP Community topic page

DOCUMENTATION

- ABAP Security in ABAP Keyword Documentation

SAP COMMUNITY

- SAP Security Community

BLOGS

- SAP Code Vulnerability Analyzer Checks
- SAP Code Vulnerability Analyzer - FAQs
- Remote code analysis with ABAP Test Cockpit (ATC)
- ABAP Test Cockpit (ATC) in the Cloud – What is already possible

Thank you.

Contact information:

Olga Dolinskaja
olga.dolinskaja@sap.com

© 2025 SAP SE or an SAP affiliate company. All rights reserved. See Legal Notice on www.sap.com/legal-notice for use terms, disclaimers, disclosures, or restrictions related to this material.

Bring out your best.